Exploring, visualizing and processing data with Python

Python Programming for Business Intelligence

Sippo Rossi
April 15th 2026

Learning goals

Theoretical:

Understanding why exploratory data analysis is done

Understand how to effectively inspect data visually

Understand what typical preprocessing of data is needed

Practical:

Know how to do exploratory data analysis

Know several basic ways to preprocess data

Motivation

Often you will need to inspect data both numerically and visually right after loading it

Especially "quick and dirty" visualizations can reveal potential problems in the data or give ideas for running more sophisticated analyses

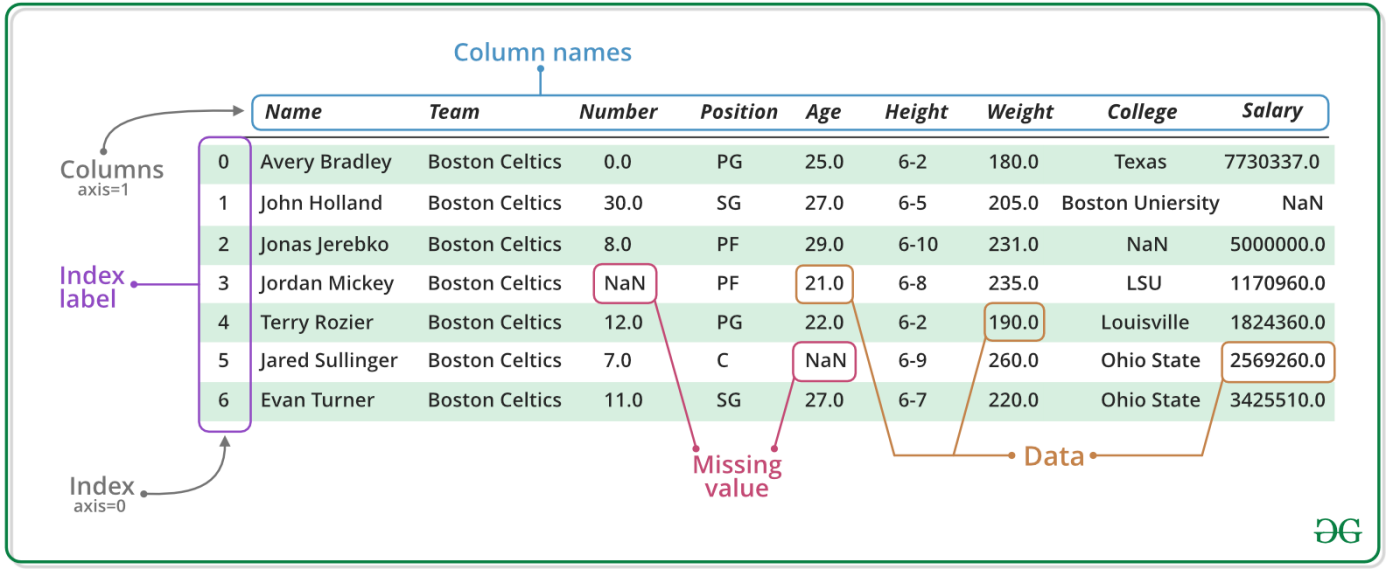
In this session we will look at how you can quickly inspect as well as clean and preprocess data so that is ready for its intended use

Agenda

- Terminology
- Exploratory data analysis
- Visualizing data
- Preprocessing data

Terminology

Pandas DataFrames



	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston Uniersity	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Image from GeeksForGeeks

Variable types

Categorical (also nominal)

- Set of values without inherent order
- Examples: gender (M, F), hair color (blond, brown, red)

Ordinal

- Set of ordered values
- Magnitude between successive values not known
- Example: clothing size (XS, S, M, L, XL)

Continuous (also numeric)

- Integer or real values
- Examples: temperature, year

Discrete

The Palmer Penguins dataset

This week's slides mainly use the dataset called Palmer Penguins, which consists of observations on Antarctic penguins on selected islands

Why? Because penguins are cool and the dataset works very nicely for basic visual demonstrations

https://archive.ics.uci.edu/dataset/690/palmer+penguins-3

Exploratory Data Analysis

Exploratory data analysis

Exploratory data analysis (EDA) is a crucial step when working with data

It refers to inspecting data visually and numerically to summarize and understand its main characteristics

EDA includes checking descriptive statistics and creating basic visualizations that help inspect and understand the data

The business intelligence process visualized

Recall this slide from last week

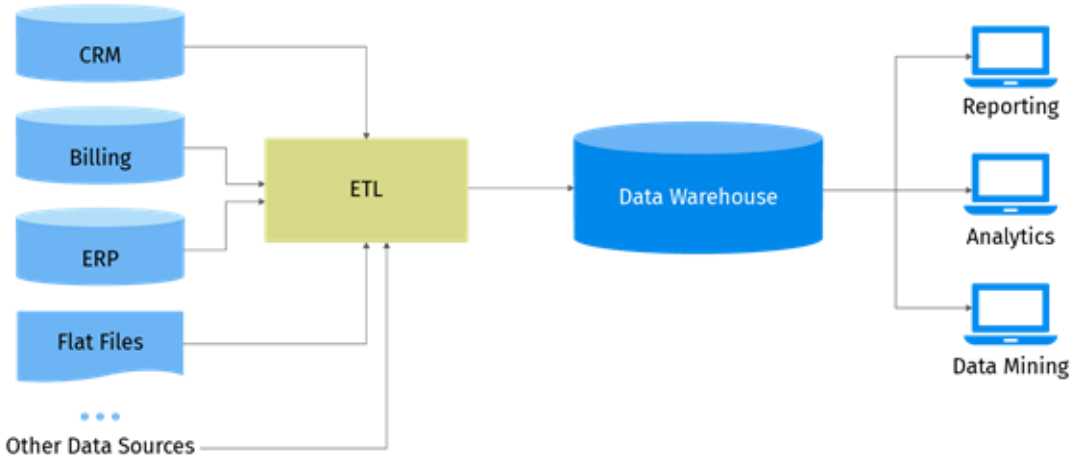


Image from: https://blog.griddynamics.com/edw-performance-comparison/

The business intelligence process visualized

EDA is done whenever you start working with a new dataset

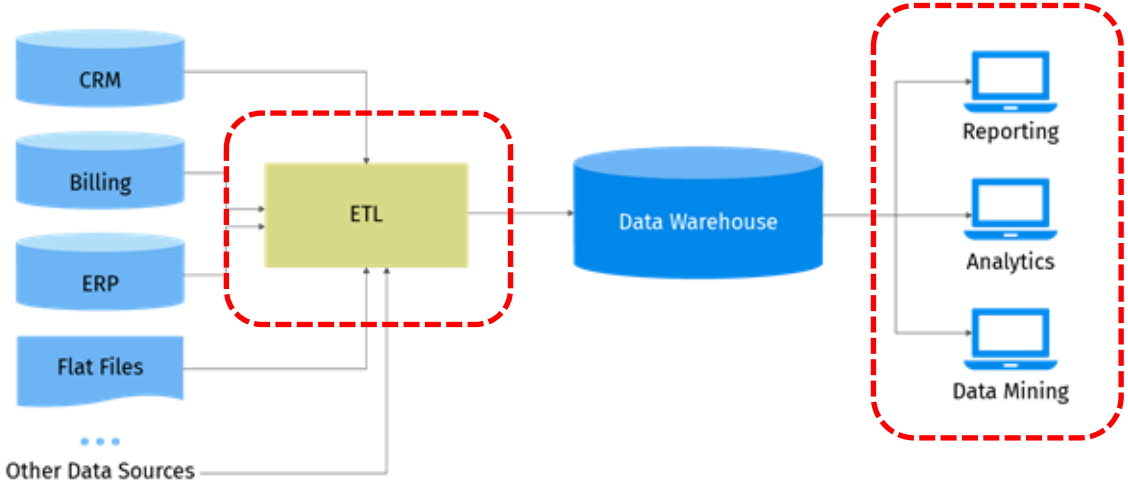


Image from: https://blog.griddynamics.com/edw-performance-comparison/

Exploratory data analysis

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
0	Adelie	Torgersen	39.100	18.700	181.000	3750.000	male	2007
1	Adelie	Torgersen	39.500	17.400	186.000	3800.000	female	2007
2	Adelie	Torgersen	40.300	18.000	195.000	3250.000	female	2007
3	Adelie	Torgersen	nan	nan	nan	nan	NaN	2007
4	Adelie	Torgersen	36.700	19.300	193.000	3450.000	female	2007

It is impossible to fully understand the data just by looking at a table

However, typically it is a good idea to inspect the first few rows to get a quick overview and to confirm that the data loaded correctly

```
df.head()
```

Exploring data numerically

Summary statistics (mean, median, mode, variance, standard deviation and range) can give a quick overview and be used to formulate new hypotheses.

Additionally, with categorical data you could check how many examples of each label there are in the data

```
df.describe()
```

```
df[feature].value_counts()
```

It is good to also check the data types

```
df.info()
```

```
df.nunique()
```

Data types and summary statistics

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    object
 1   island             344 non-null    object
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    object
 7   year               344 non-null    int64
dtypes: float64(4), int64(1), object(3)
memory usage: 21.6+ KB
```

```
df.info()
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	342.000	342.000	342.000	342.000	344.000
mean	43.922	17.151	200.915	4201.754	2008.029
std	5.460	1.975	14.062	801.955	0.818
min	32.100	13.100	172.000	2700.000	2007.000
25%	39.225	15.600	190.000	3550.000	2007.000
50%	44.450	17.300	197.000	4050.000	2008.000
75%	48.500	18.700	213.000	4750.000	2009.000
max	59.600	21.500	231.000	6300.000	2009.000

```
df.describe()
```

Exploring data visually

Visual inspection is an important part of EDA

Summary statistics alone can be deceiving

All datasets on the right have identical summary statistics

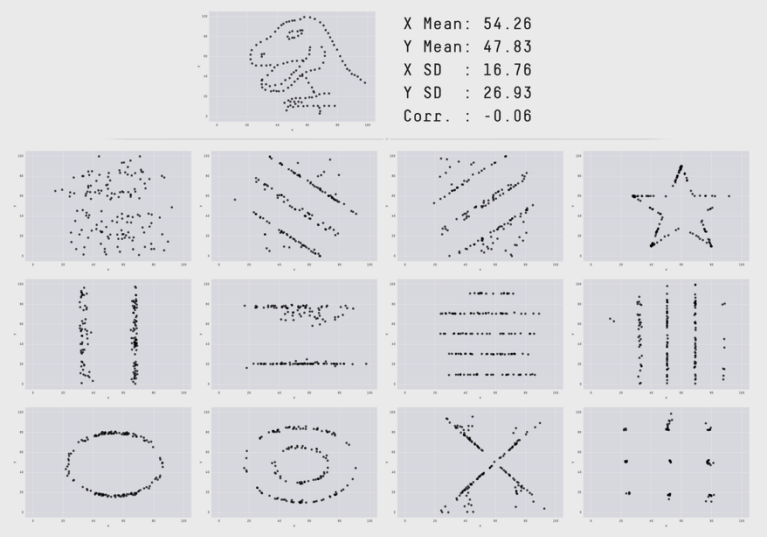
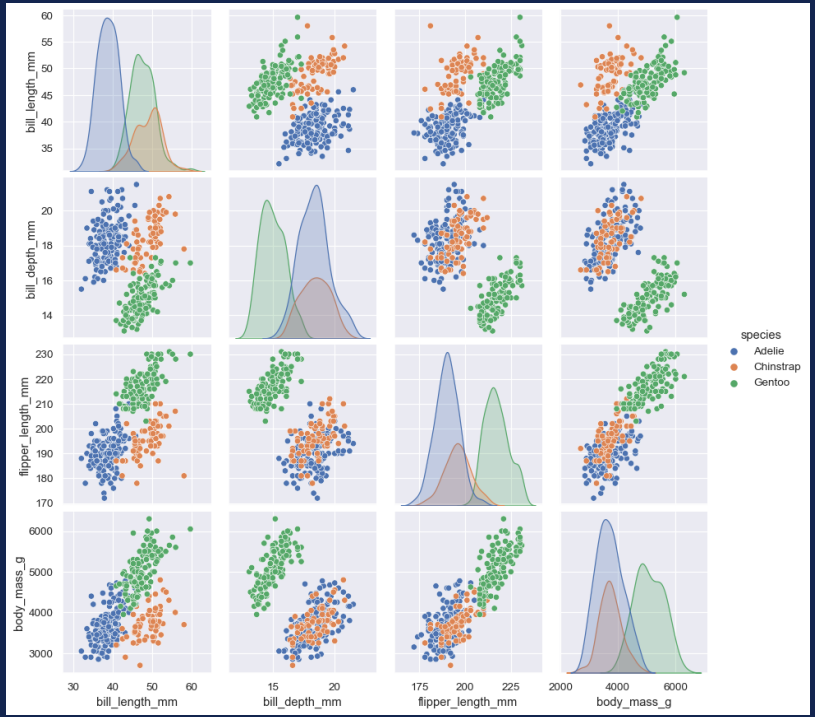


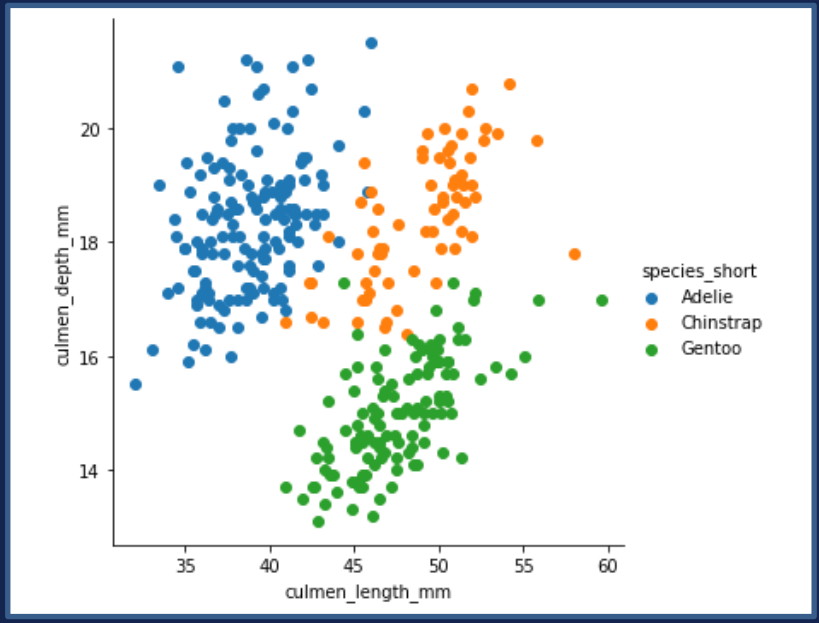
Image from: http://dx.doi.org/10.1145/3025453.3025912

Exploring data visually




```
sns.pairplot(df, hue="species_short")
```

Exploring data visually




```
sns.FacetGrid(df, hue="species_short", height=5).map(plt.scatter, "culmen_length_mm", "culmen_depth_mm").add_legend()
```

Visualizing data

Matplotlib

Matplotlib is the "standard" plotting library for Python

Matplotlib visualizations are extremely customizable, but consequently the syntax is somewhat complicated and making relatively basic plots pretty requires many, many lines of code

Many other Python visualization libraries are built on top of Matplotlib

Documentation and more information can be found here:
https://matplotlib.org

Matplotlib

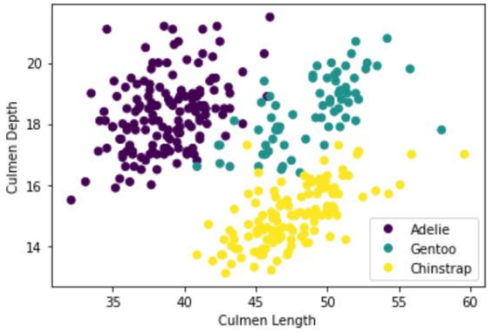
```
plt.figure()

species = ['Adelie', 'Gentoo', 'Chinstrap']
scatter = plt.scatter(df.culmen_length_mm, df.culmen_depth_mm,
                      c=df.species_short.astype('category').cat.codes)

# Adding x and y labels
plt.xlabel("Culmen Length")
plt.ylabel("Culmen Depth")

# add legend to the plot with names
plt.legend(handles=scatter.legend_elements()[0],
           labels=species, loc = 'lower right')
```

```
<matplotlib.legend.Legend at 0x7ff34db8aeb0>
```



Seaborn

Seaborn is a higher-level library based on Matplotlib, which allows making prettier plots with fewer lines of code

Seaborn works well with pandas DataFrames and arrays as well as with NumPy arrays and standard Python data structures such as lists and dictionaries

This week's examples and exercises will use seaborn

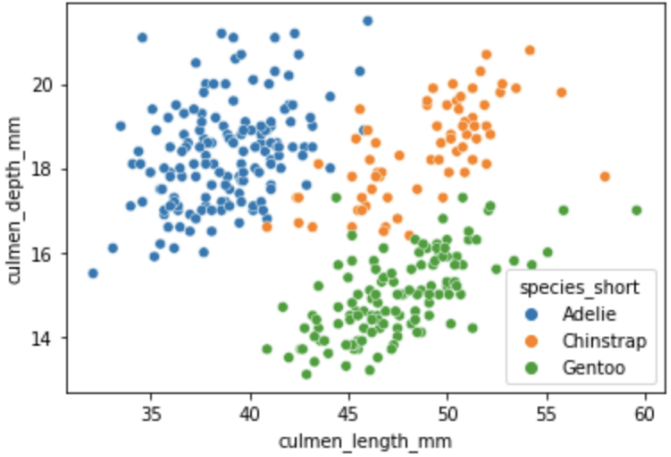
Seaborn is imported typically as sns

Why is it called seaborn and why do we import it as sns? It is an obscure reference to a character's name in an old tv show

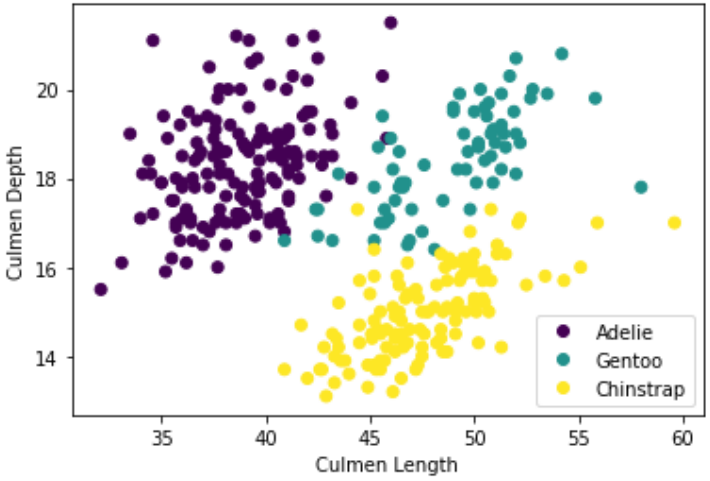
Seaborn

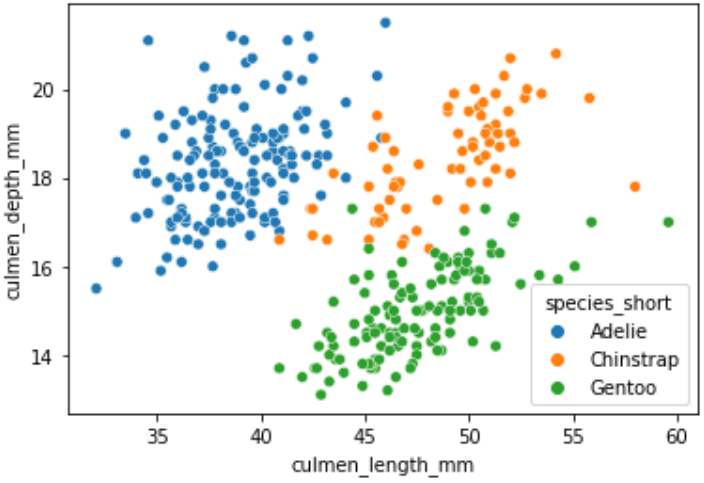
```
sns.scatterplot(data = df, x="culmen_length_mm", y="culmen_depth_mm",
                hue = "species_short")
```

```
<AxesSubplot:xlabel='culmen_length_mm', ylabel='culmen_depth_mm'>
```



Matplotlib with 5 lines of code vs seaborn with 1





How seaborn works

Seaborn is used mainly through different plotting functions

You specify the plot type by using a function such as "histplot" or "scatterplot" and then through parameters give specify the data and style. E.g.:

```
sns.histplot(data=df, x="flipper_length_mm", hue="species_short", multiple="stack")
```

Note how you can access the columns of the DataFrame just by referring to them by the column name.

A note on workflows when visualizing data

A tedious part of making visualizations, especially with highly customizable libraries such as Matplotlib, has been understanding how different parameters work

Many prefer to copy paste existing code examples and adjusting it to fit their data, rather than building from scratch. E.g., seaborn provides a nice gallery of examples with code

More recently generative AI has somewhat changed the process of making visualizations. Especially more advanced models such as Claude Opus 4.6 are getting good at adjusting visualizations simply based on textual descriptions

Preprocessing data

Preprocessing data

Most of the time data requires some preprocessing and or cleaning

Issues can be known in advance or noticed during exploration

Common issues:

- Missing values
- Faulty data
- Wrong data type
- Duplicates

	species_short	island	culmen_length_mm	culmen_depth_mm
0	Adelie	Torgersen	39.100	18.700
1	Adelie	Torgersen	39.500	17.400
2	Adelie	Torgersen	40.300	18.000
3	Adelie	Torgersen	nan	nan
4	Adelie	Torgersen	36.700	19.300

Preprocessing data

Some examples of ways to clean and preprocess data:

- Convert fields with text into numeric form
- Remove rows with missing / faulty data
- Fix rows with faulty datapoints
- Drop unnecessary variables (columns)
- Scaling or normalizing data
- Creating new variables
- Renaming variables (remove capitalization, spaces)

Removing values vs imputing

Imputing means replacing missing data with substituted values

Whether you should remove or impute data depends on the situation

Removing missing values is in general less risky than imputing

But removing missing values could make the dataset too small or imbalanced (if errors are mainly on similar types of rows)

Consider removing variables with lots of missing values rather than rows

Removing missing values

	species_short	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.100	18.700	181.000	3750.000	MALE
1	Adelie	Torgersen	39.500	17.400	186.000	3800.000	FEMALE
2	Adelie	Torgersen	40.300	18.000	195.000	3250.000	FEMALE
3	Adelie	Torgersen	nan	nan	nan	nan	NaN
4	Adelie	Torgersen	36.700	19.300	193.000	3450.000	FEMALE

	species_short	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.100	18.700	181.000	3750.000	MALE
1	Adelie	Torgersen	39.500	17.400	186.000	3800.000	FEMALE
2	Adelie	Torgersen	40.300	18.000	195.000	3250.000	FEMALE
4	Adelie	Torgersen	36.700	19.300	193.000	3450.000	FEMALE
5	Adelie	Torgersen	39.300	20.600	190.000	3650.000	MALE

In this example we drop all rows with missing data

```
df.dropna()
```

Imputing missing values

Univariate imputation:

- use non-missing values of the same feature (column) to give a substitute for missing values
- e.g. replacing the missing value with the the mean, mode or median of the feature

```
SimpleImputer()
```

Other alternative include multivariate imputation and nearest neighbor imputation. Read more here

Imputing missing values

	species_short	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	0	2	39.1	18.7	181.0	3750.0	2
1	0	2	39.5	17.4	186.0	3800.0	1
2	0	2	40.3	18.0	195.0	3250.0	1
3	0	2	NaN	NaN	NaN	NaN	3
4	0	2	36.7	19.3	193.0	3450.0	1
...	...	...	...	...	...	...	...
339	2	0	NaN	NaN	NaN	NaN	3

	species_short	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	0	2	39.1	18.7	181.0	3750.000000	2
1	0	2	39.5	17.4	186.0	3800.000000	1
2	0	2	40.3	18.0	195.0	3250.000000	1
3	0	2	NaN	NaN	NaN	4201.754386	3
4	0	2	36.7	19.3	193.0	3450.000000	1
...	...	...	...	...	...	...	...
339	2	0	NaN	NaN	NaN	4201.754386	3

```
df["body_mass_g"] = imp.fit_transform(df[["body_mass_g"]])
```

Normalization and standardization

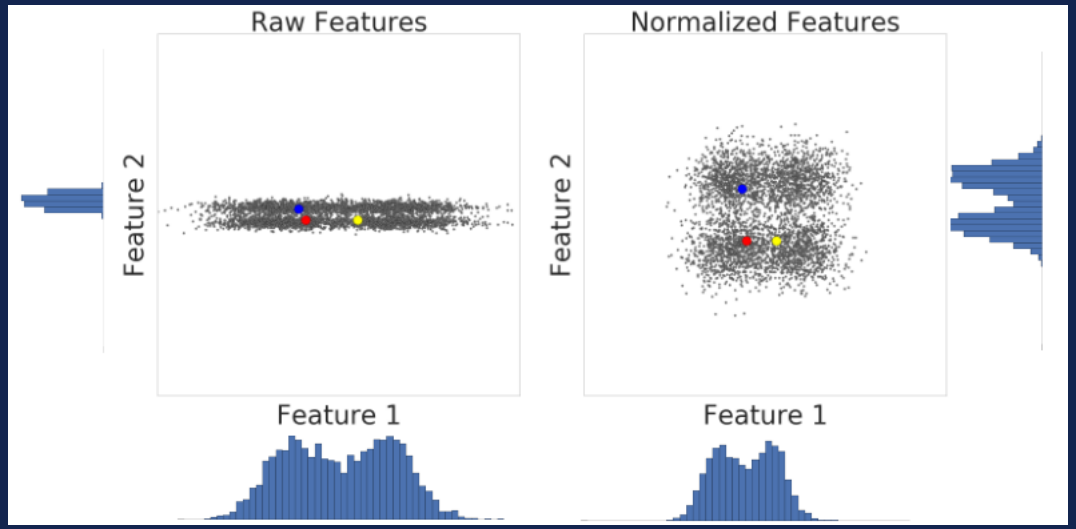
In some situations normalizing (scaling) and standardizing data is beneficial. For example, some statistical methods and especially machine learning algorithms are sensitive to scale or assume normality

E.g. one variable is in the range of 0-1 while another 100-10000

To mitigate such issues data can be

- normalized (e.g. from 0 to 1) `sklearn.preprocessing.MinMaxScaler()`
- standardized (data points are expressed as σ from the mean)
- log transformed `sklearn.preprocessing.StandardScaler()`

An example of normalization




```
sklearn.preprocessing.StandardScaler()
```

Image from https://developers.google.com/machine-learning/clustering/prepare-data

Creating new variables

Sometimes it may be meaningful to combine features into one feature by creating a ratio of the two

The new feature can provide more or equal information as the previous separate columns of data

Examples:

- Often on social media accounts have followers and people that the account follows. A follower to following ratio can help to determine if an account is e.g. an influencer, likely to be a bot or a regular user
- Combining weight and height to BMI

EDA and preprocessing example

Going through *L4_EDA_and_preprocessing.ipynb*

Conclusion

Conclusion

Exploratory data analysis combines both numeric and visual inspection of data

Visualizations often can give us an overview of how the data is distributed and if there are correlations between variables among other things

Often some form of pre-processing is needed before the data is ready to work with

Next week we will focus on more detailed statistical and visual analysis, as well as on making high quality plots

Questions, comments suggestions?